编译原理
11. 寄存器分配

rainoftime.github.io
浙江大学
计算机科学与技术学院

Content

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. **Register Allocation**
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

Outline

Introduction to Register Allocation

Register Allocation via Graph Coloring

Coloring by Simplifications

Coloring by Simplifications, Cont.

4. Coloring By Simplifications. Cont.

- Coalescing
- Precolored Nodes
- Putting it all Together

Recap: Special Handling of MOVE

- When an instruction **defines** variable *a* and the live-out set is $\{b_1, \ldots, b_j\}$:

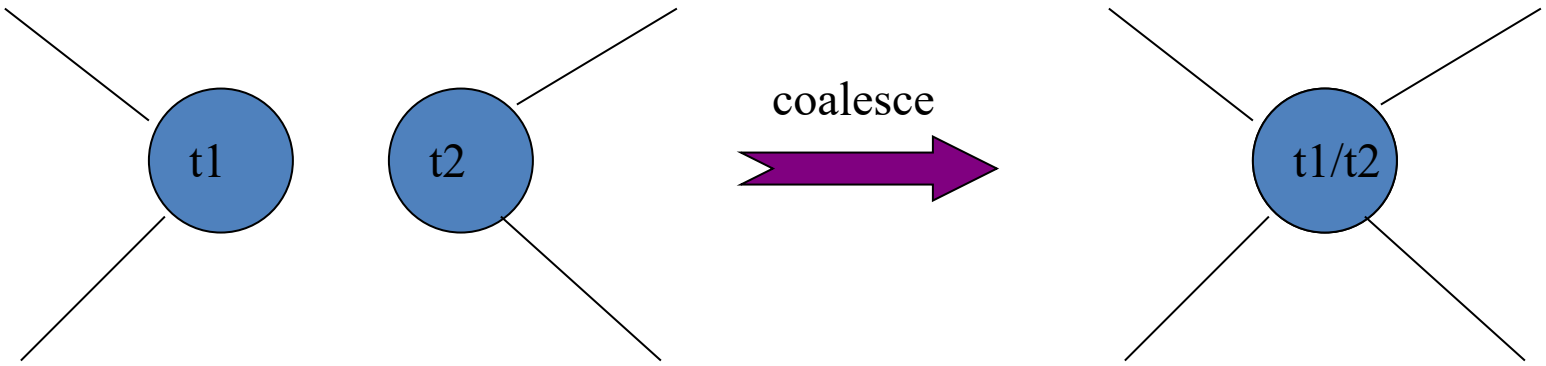
Instruction type	**Interference edges added**
Non-MOVE (e.g., a ← b+c)	$(a, b_1), (a, b_2), \ldots, (a, b_j)$ — all live-outs
MOVE (a ← c)	(a, b_i) for every $b_i \neq$ **c**

For a MOVE a ← c, we do **not** add an edge (a, c). This keeps open the possibility that a and c can share the same register — enabling coalescing!

- Additionally, we record a **move edge** (dashed line) between a and c to mark them as *coalescing candidates*

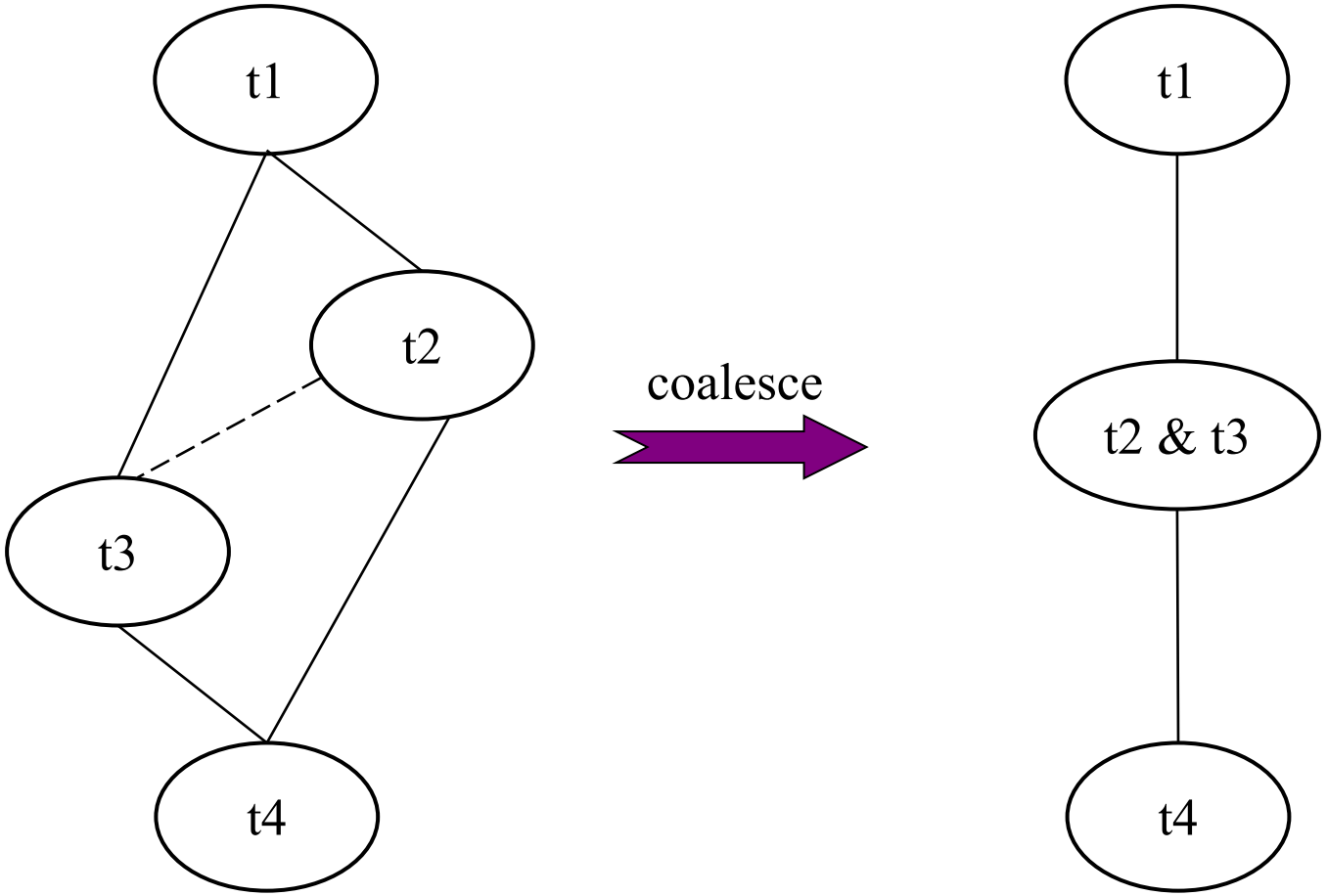
What is Coalescing

- Given a MOVE instruction: MOVE a, b
- If variables a and b don't interfere
  1. The move instruction can be eliminated, and
  2. a, b are coalesced into a new node, whose edges are the union of those of the nodes being replaced.



Why Coalescing Helps

- Coalescing may **improve the coloralibility**

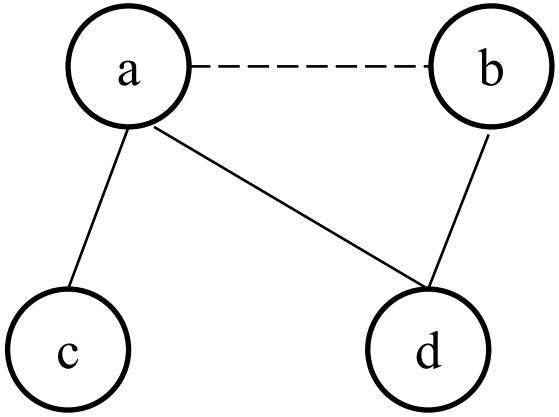


After coalescing, t1 and t4 have one neighbor less!

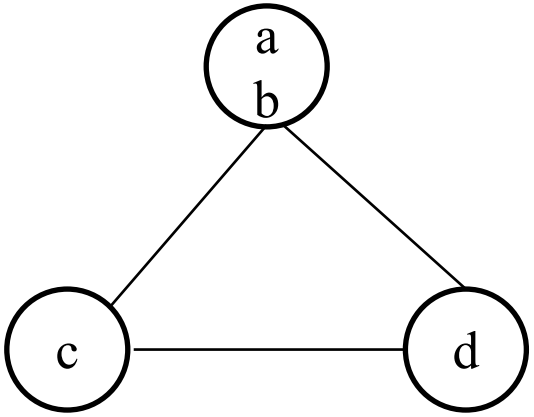
Why Not Coalescing

- **Problem**: Coalescing may increase the number of interference edges and make a graph ***uncolorable***!

Before (K=2, 2-colorable):



✗ NOT 2-colorable



- **Idea**: Conservative coalescing: don't make it harder，i.e., coalesce a and b guided by heuristics

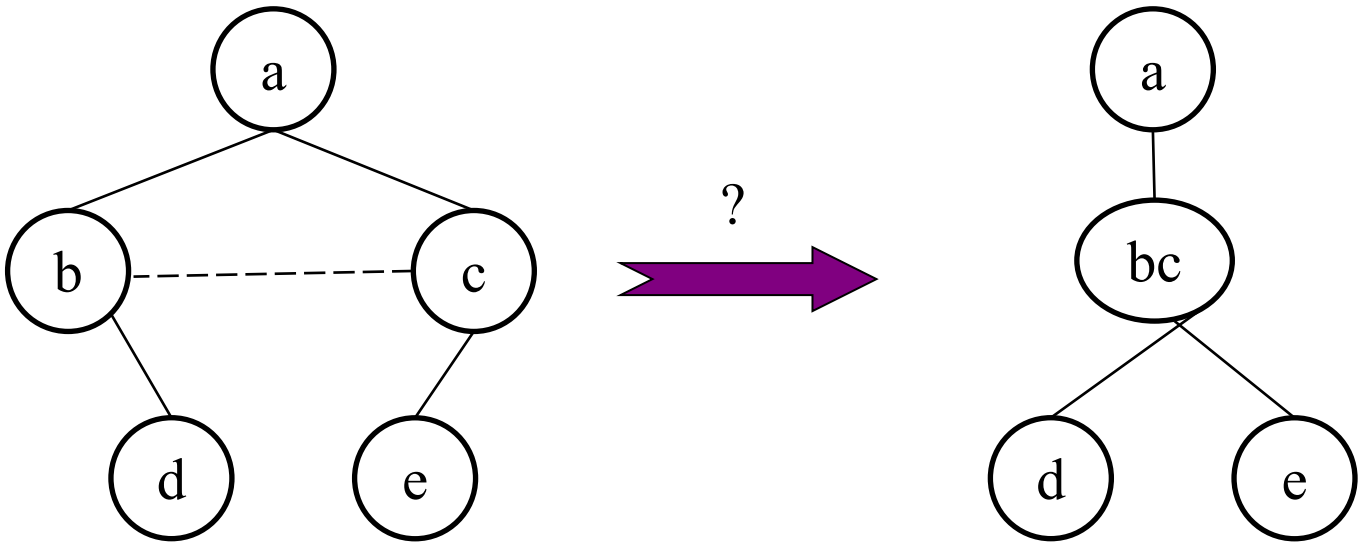
Briggs's Criteria

- **Briggs Criteria**: Nodes a and b can be coalesced if the resulting node ab will have fewer than K neighbors of significant degree (degree ≥ K).
- **Intuition:**
  - After coalescing, can ab eventaully be simplified?
  - If ab has < K high-degree neighbors → ab is *not* a problematic node
  - The low-degree neighbors can be simplified away first, eventually making ab low-degree too

Safety guarantee: If the graph was K-colorable before, it remains K-colorable after coalescing under Briggs's criterion.

Example: Briggs Criterion (K=3)

- Nodes a and b can be coalesced if the resulting node ab will have fewer than K neighbors of significant degree

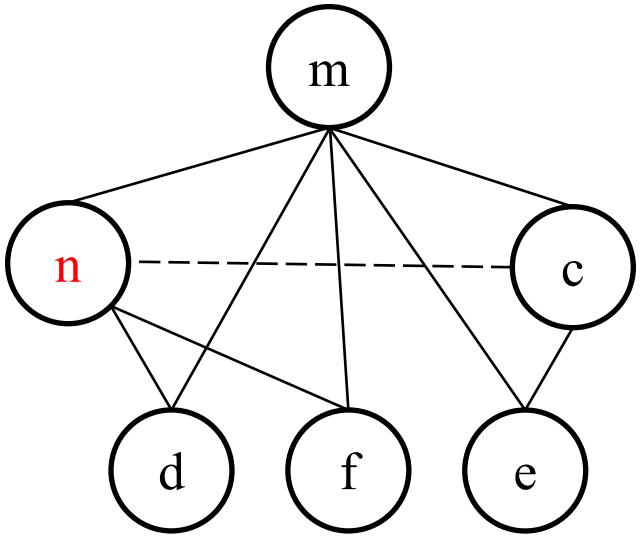


George Criterion

- **George Criteria**: Nodes a and b can be coalesced if for **every neighbor** t of a, either
  1. t already **interferes with b** or
  2. t is of **insignificant degree** (< K)
- **Intuition:**
  - Merging a into b should not create any *new* problematic constraints
  - If t already interferes with b → merging adds *no new edge* for t
  - If t has low degree → even with a new edge, t is still simplifiable

Example: George Criterion (K=3)

- Nodes a and b can be coalesced if for every neighbor t of a, either
  1. t already interferes with b or
  2. t is of insignificant degree (< K)



- n has neighbors: m, d, f
- For each neighbor of n, check
  1. m: interferes with c ✓
  2. d: low degree ✓
  3. f: low degree ✓

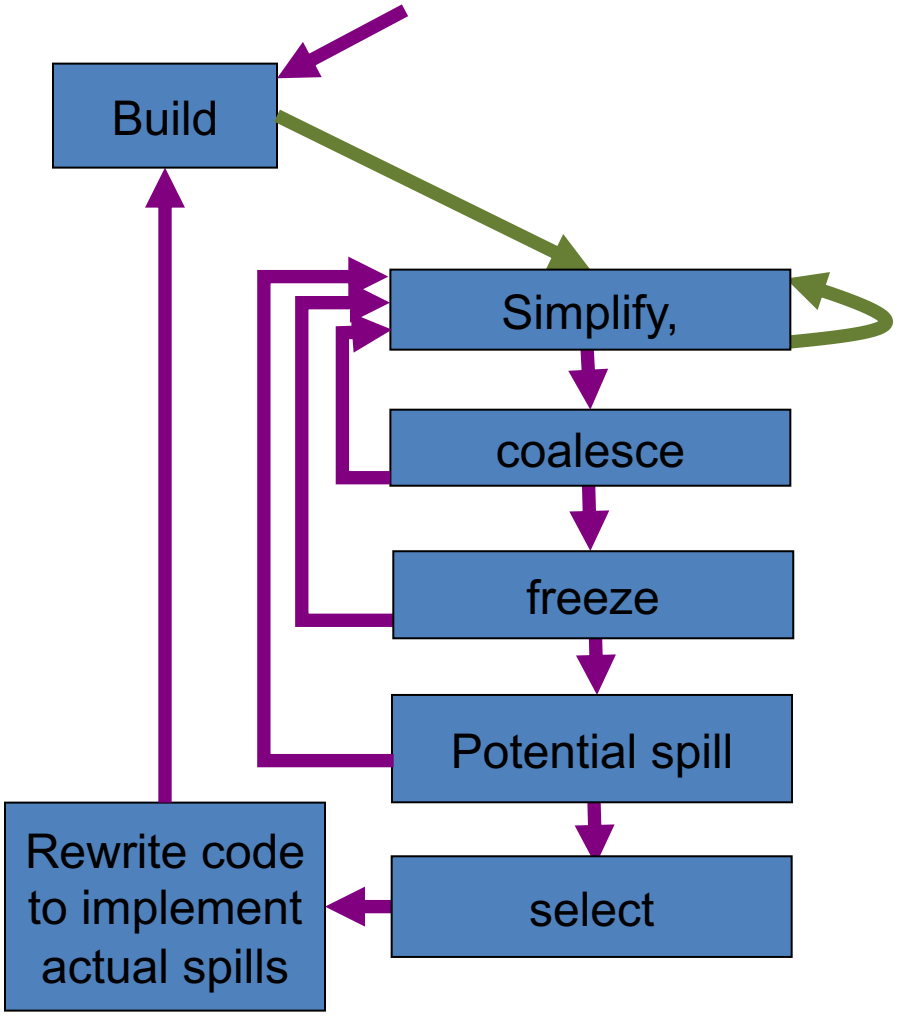
Yes - all neighbors of n either interfere with c or have low degree

Briggs vs. George

	Briggs's Criterion	**George's Criterion**
Approach	Count high-degree neighbors of merged node	Check each neighbor of one node individually
Perspective	Global: "Is the merged node OK?"	Local: "Does merging harm any neighbor?"
Best for	Two ordinary temporaries	Moves involving precolored nodes
Weakness	Too conservative with infinite-degree nodes	May miss cases where neighbors share edges

In practice: **Briggs** for ordinary-ordinary moves, **George** for moves involving precolored nodes (whose degree = ∞ would make Briggs too conservative).

Optimized Algorithm with MOVE Coalescing



1. **Build** interference graph (and add move edges for MOVE)
2. **Simplify**: Remove non-move-related nodes of low-degree
3. **Coalesce :**Apply conservative coalescing
4. **Freeze:** If no more coalescing is possible, convert move edges to regular edges!
5. **Potential Spill:** If necessary, choose nodes for potential spills
6. **Select:** Assign colors to nodes
7. **Actual Spill**: If select failed, rewrite code to implement actual spill and rerun the whole loop

Optimized Algorithm with MOVE Coalescing

Phase	Action	Key Point
1. Build	Construct interference graph + move edges	Classify nodes as move-related or not
2. Simplify	Remove **low-degree non-move-related** nodes	Push onto stack (only non-move-related!)
3. Coalesce	Apply conservative coalescing (Briggs/George)	Merged node → non-move-related → back to Simplify
4. Freeze	Give up coalescing for a low-degree move-related node	Marks it non-move-related → back to Simplify
5. Potential Spill	Choose a high-degree node for potential spilling	Push onto stack, continue Simplify
6. Select	Pop stack, assign colors	May discover actual spills
7. Actual Spill	Rewrite code with load/store, rebuild graph	Restart from Build

Coloring with Coalescing

- The coalesce, simplify, and spill procedures should be alternated until the graph is empty

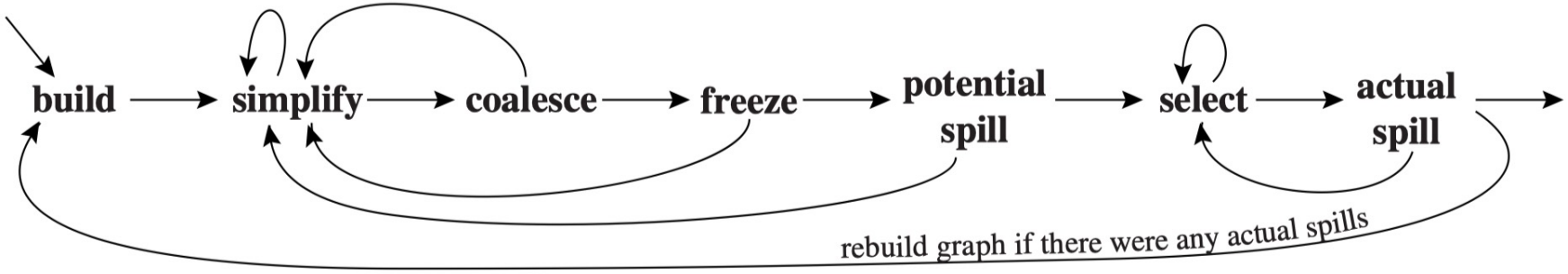


FIGURE 11.4. Graph coloring with coalescing.

Coloring with Coalescing

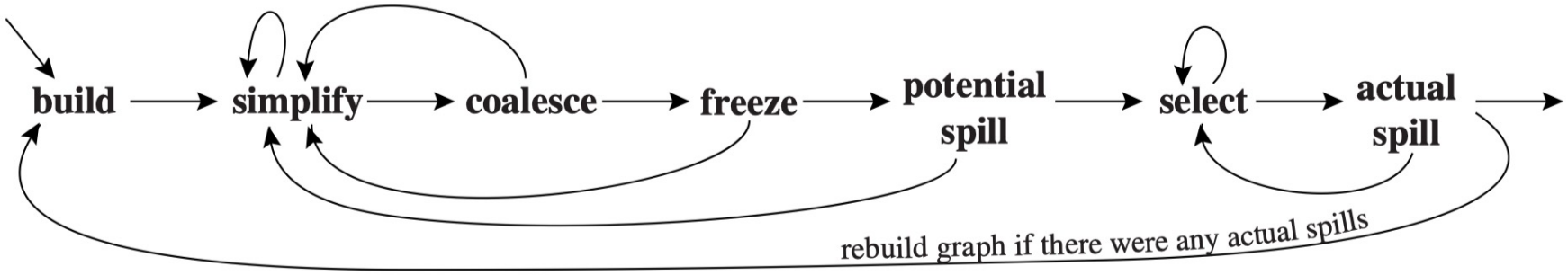


FIGURE 11.4. Graph coloring with coalescing.

1. Build

- Construct the interference graph
- Add move edges (dashed) for MOVE instructions
- Categorize each node as move-related or non-move-related

Coloring with Coalescing

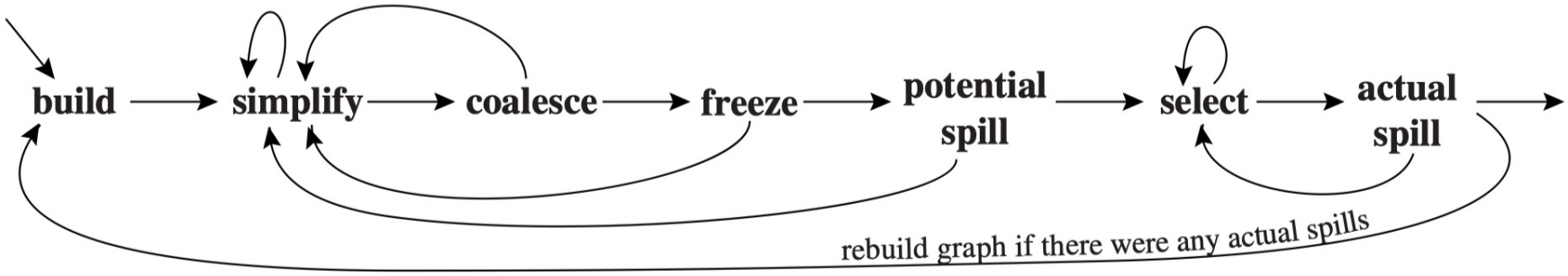


FIGURE 11.4. Graph coloring with coalescing.

2. Simplify

- One at a time, remove **non-move-related** nodes of low (<K) degree from the graph

Important change from basic algorithm:

We only simplify **non-move-related** nodes!

Move-related nodes are held back for potential coalescing.

Coloring with Coalescing

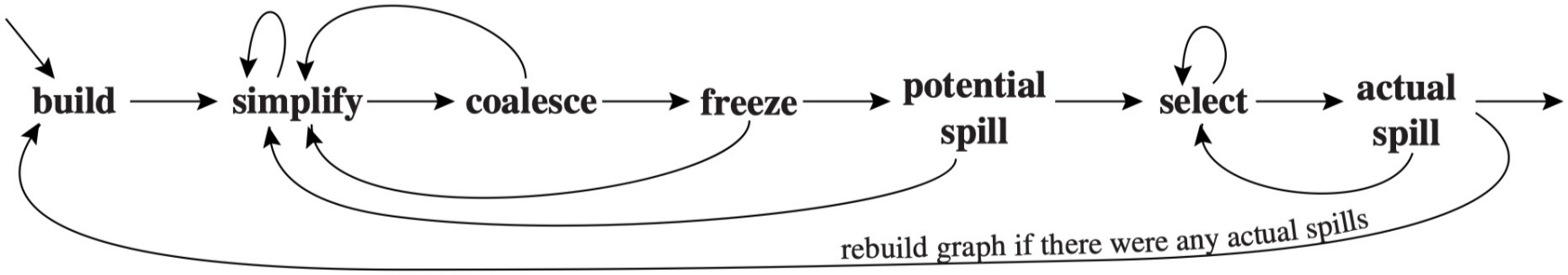


FIGURE 11.4. Graph coloring with coalescing.

3. Coalesce

- Conservative coalescing on the reduced graph
- The resulting node is no longer move-related → available for the next round of simplification

- Simplify and coalesce are repeated until
  - only significant-degree
  - or move-related nodes remain

Coloring with Coalescing

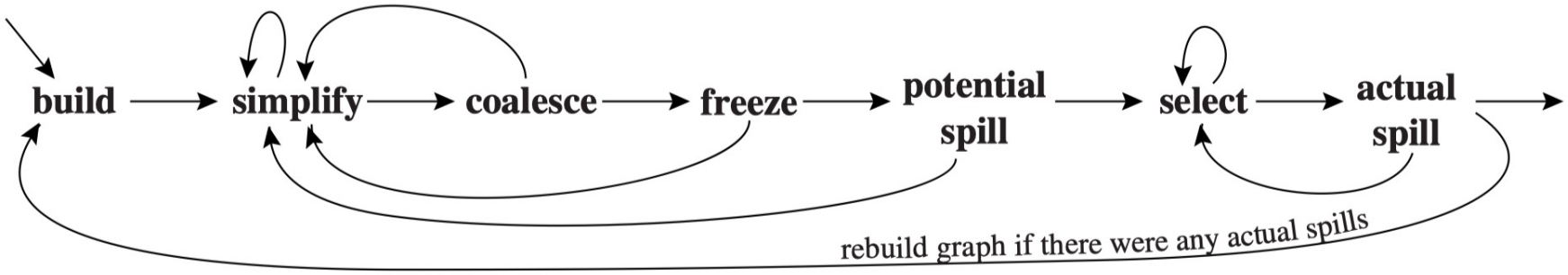


FIGURE 11.4. Graph coloring with coalescing.

4. Freeze ❄

- If neither simplify nor coalesce applies, look for a move-related node of low degree.
- We **freeze** the moves: giving up the hope of coalescing those moves.
- Those **nodes are considered non-move-related** 😄
- Benefit: Simplify and coalesce are resumed!

Coloring with Coalescing

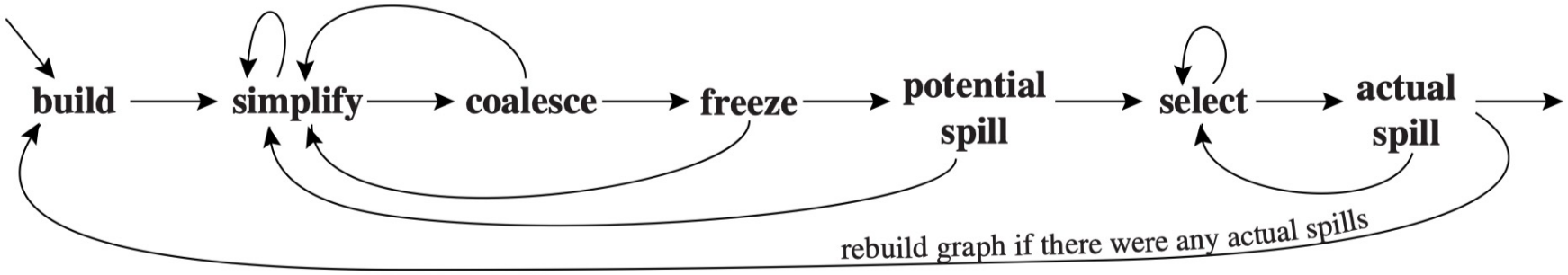


FIGURE 11.4. Graph coloring with coalescing.

5. (Potential) Spill

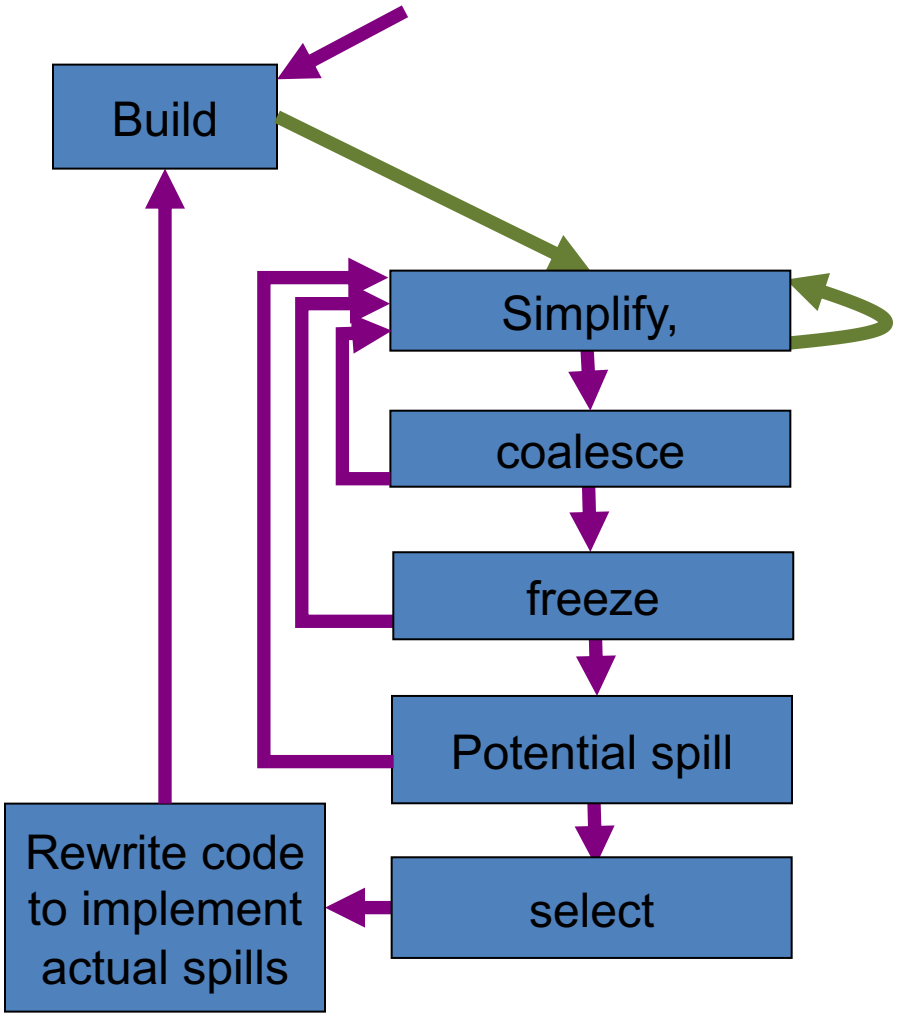
- If there are no low-degree nodes, we select a significant-degree node for *potential spilling*

6. Select

- Pop the entire stack, assigning colors.

7. Rebuild graph if there are any actual spills!

Optimized Algorithm with Move Coalescing

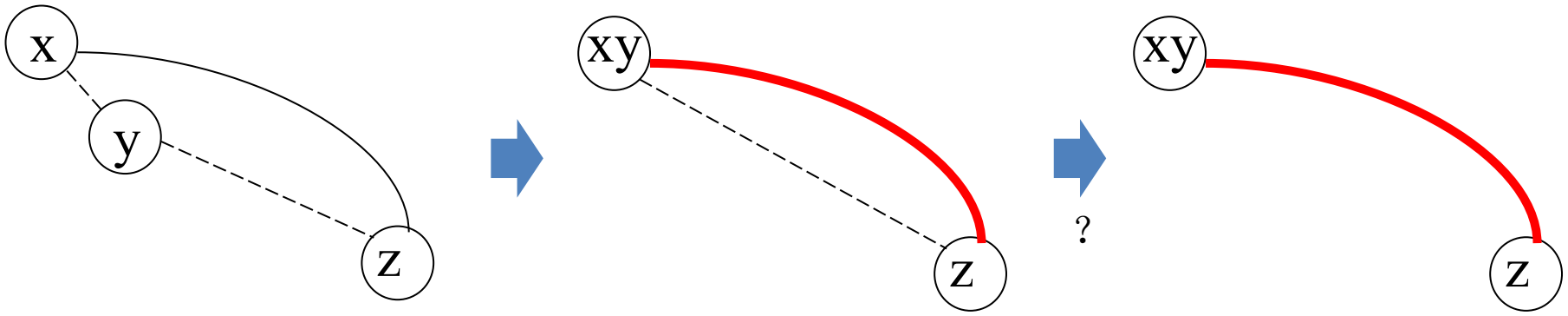


1. **Build: interference graph + move edges**
2. **Simplify**: 先做simplify，能做就做，但是只能simplify non-move-related node
3. **Coalesce :** 尝试合并a和b，如果可以的话，那么合完以后{a, b}就变成了non-move-related node，继续回去simplify
4. **Freeze:** 如果没有东西可以coalesce，那么就去freeze，不合并这两个节点了。Freeze后可以继续simplify
5. **Potential Spill:**如果图上还有东西，所以所有的节点都是significant node，这时候我们就要spill了，此时是标记为potential spill 。继续回去simplify，直到图为空
6. **Select:** 退栈着色过程
7. **Actual Spill**:如果标记为了potential spill，到了某个节点可能没有办法着色了，此时我们就真的去做actual spill当前的这个node，重新写这个node的load和store信息

此时liveness的信息就会发生变化。我们要重新构造RIG图。然后再回来跑一遍算法

One More Thing: Constrained Moves

- **Constrained Move:** a move instruction that cannot be coalesced because the source and destination interfere
- E.g.: two moves x ← y and y ← z, and an interference edge (x, z)



  - If we coalesce x and y first, the resulting node x&y will interfere with z, making the second move (y ← z) constrained.
  - Because of (xy, z) are interfered, we can **freeze** it

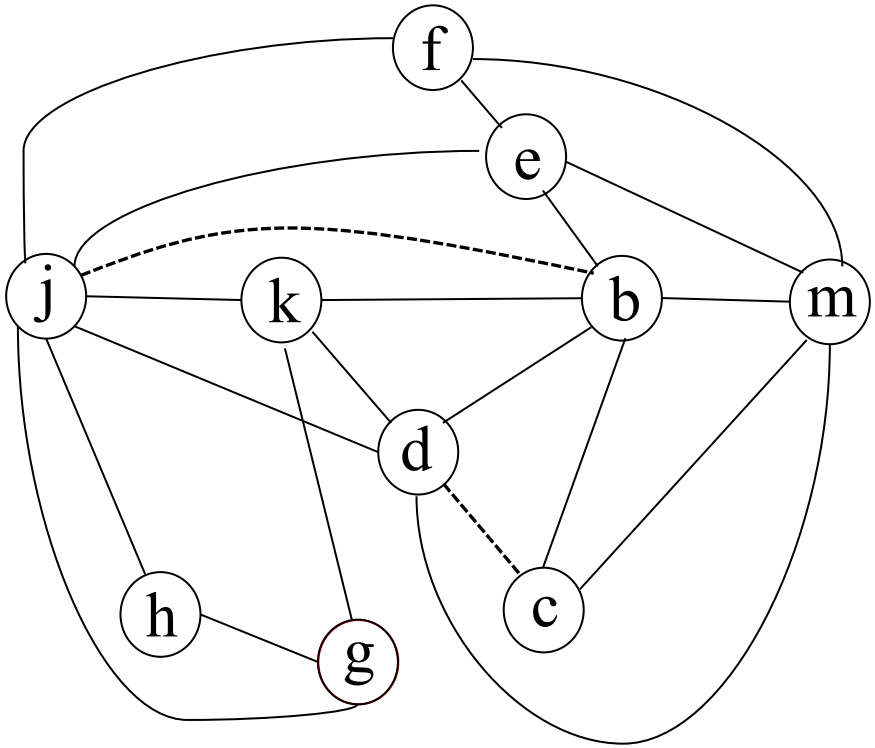
识别出因冲突而不可能合并的 move 指令，将它们排除出合并候选集，从而让相关节点可以被simplify处理，算法持续向前推进

Example (K=4), Revisited

Live in: k j

```
g := mem[j+12]
h := k-1
f := g*h
e := mem[j+8]
m := mem[j+16]
b := mem[f]
c := e+8
d := c
k := m+4
j := b
```

Live out d k j



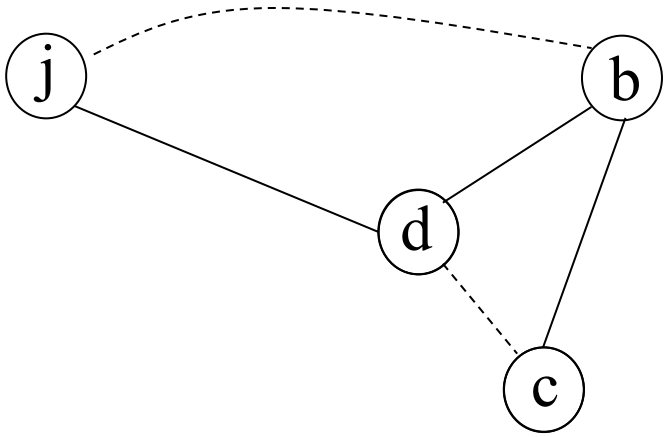
Step 1: Simplify g, h, k, f, e, m

- **We first simplify the non-move-related, low-degree nodes: g, h, k, f, e, m**

Stack

m
e
f
k
h
g

"Simplify" phase cannot continue here:
(It only works for non-move-related nodes)



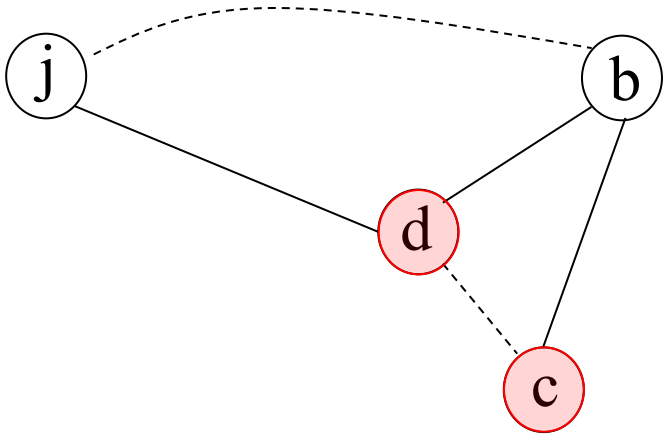
Step 2: Coalescing c&d

- **We then check for coalescing opportunities**

  ➔ Find that we can coalesce c and d (Brigs criteria)

Stack

m
e
f
k
h
g



Step 3: Simplify c&d

- **After merging c&d, we can continue simplify the the merged node (it is non-move-related)!**

Stack

m
e
f
k
h
g

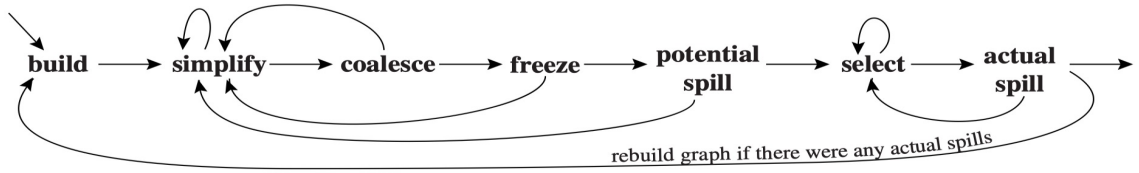
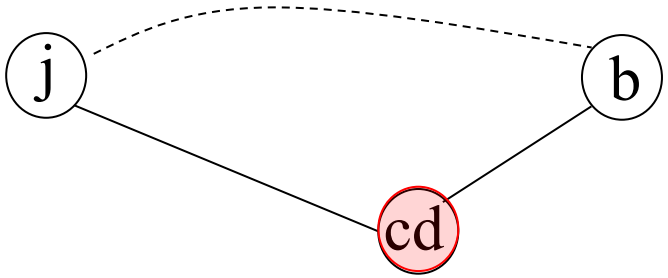


FIGURE 11.4. Graph coloring with coalescing.



此时cd就变成了non-move-related node，可以被拿走

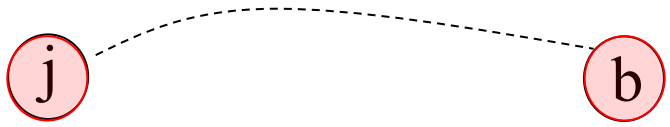
- Simplify and coalesce are repeated until
  - only significant-degree
  - or move-related nodes remain

Step 4: Coalescing b&j

- **We cannot simplify now**😭 (only move-related nodes)

Stack

cd
m
e
f
k
h
g



Step 4: Coalescing b&j

- **We cannot simplify now** 😭 (only move-related nodes)
- **We then check for coalescing opportunities**

  ➔ Further find that b and j are coalesceable (Briggs)

Stack

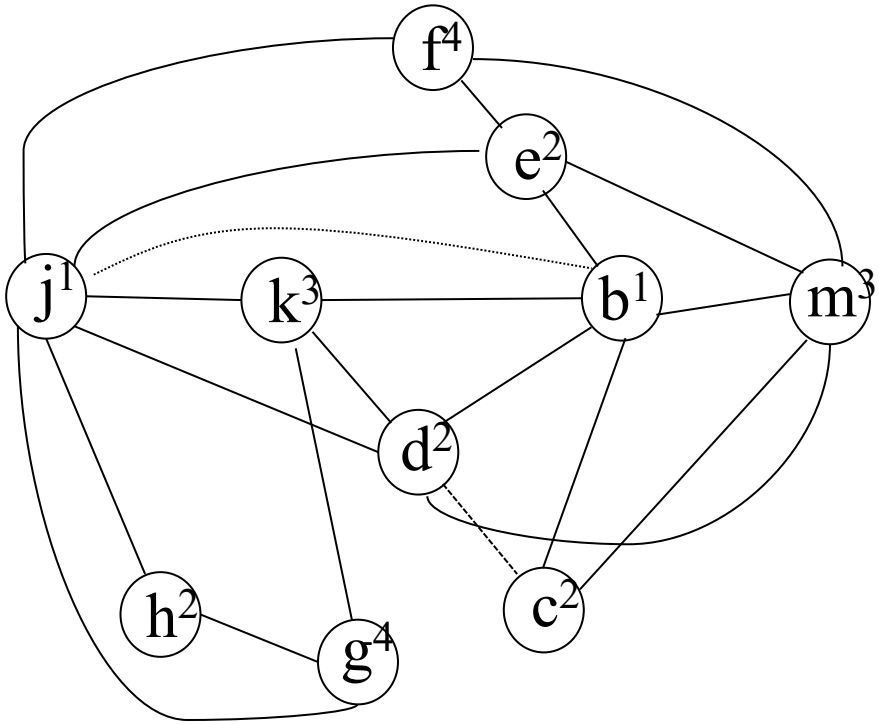
cd
m
e
f
k
h
g

Step 5: Select (Pop & Color)

- **A possible final assignment of colors**

Stack

jb	r1
cd	r2
m	r3
e	r2
f	r4
k	r3
h	r2
g	r4



Step 5: Select (Pop & Color)

```
Live in: k  j
        g := mem[j+12]
        h := k-1
        f := g*h
        e := mem[j+8]
        m := mem[j+16]
        b := mem[f]
        c := e+8
        d := c
        k := m+4
        j := b
Live out d k j
```

```
Live in: r3,r1
        r4 := mem[r1+12]
        r2 := r3-1
        r4 := r4*r2
        r2 := mem[r1+8]
        r3 := mem[r1+16]
        r1 := mem[r4]
        r2 := r2+8
        r3 := r2+4
Live out r2 r3 r1
```

The two move instructions are eliminated!!

4. Coloring By Simplifications. Cont.

- Coalescing
- Precolored Nodes
- Putting it all Together

Why Precoloring

- In a real machine, not all registers are interchangeable:
  - **Frame pointer** (e.g., %ebp) — fixed by convention
  - **Stack pointer** (e.g., %esp) — always points to top of stack
  - **Argument registers** — calling convention says $arg_1 \rightarrow$ r0, $arg_2 \rightarrow$ r1, …
  - **Return-value register** — result must be in r0 (or %eax)

The register allocator must **respect** these hardware constraints. We model them as **precolored nodes** in the interference graph.

Rules for Precolored Nodes

- In a K-register machine, there are exactly **K precolored nodes** — one for each physical register.

Property	**Explanation**
All precolored nodes **interfere with each other**	No two physical registers can share a color
Cannot be simplified	Their color is fixed — not a "hope", but a fact
Cannot be spilled	They are registers — no memory alternative
Treat as having **infinite degree**	They are never candidates for removal from the graph

Handling Precolored Nodes

1. **Build** the interference graph including precolored nodes
2. **During simplification:**
   - Never remove precolored nodes from the graph
   - Only simplify *non-precolored* (ordinary) nodes
3. **Coalesce & Spill**: continue until only precolored nodes rema
4. **During coloring:**
   - Respect the colors already assigned to precolored nodes
   - Consider interferences with precolored nodes when picking a color

Goal: Simplify, coalesce and spill until only the precolored nodes remains in the graph

Handling Precolored Nodes: Coalescing

- When coalescing a move involving a precolored node, we use **George's criterion** instead of Briggs's:
  - **George's criterion:** Nodes *a* (precolored) and *b* can be coalesced if, for every neighbor *t* of *b*, either:
    (1) *t* already interferes with *a*, or
    (2) *t* has degree < K
- **Why not using Briggs?**
  - Briggs counts high-degree neighbors of the merged node
  - Precolored nodes have *infinite* degree → Briggs would be overly conservative
  - George's criterion checks neighbors of *b* individually — safe and more permissive

Keep Live Ranges Short! (A critical front-end responsibility)

- Since precolored nodes **cannot be spilled**, the front end must keep their **live ranges short**.
  - **Problem:** A long live range of a precolored register blocks other variables from using it
  - **Solution:** Generate MOVE instructions to copy values *to/from* fresh temporaries immediately

```
// BAD: r0 live for entire function body
arg ← r0              // r0 stays live...
... long code ...
use(arg)              // ...all the way here

// GOOD: copy to fresh temp, free r0 quickly
t1 ← r0               // r0 live range = 1 instruction
... long code ...
use(t1)               // t1 can be spilled if needed
```

The MOVE from r0 to t_1 may later be **coalesced** away if register pressure is low — no overhead!

Caller/Callee-Saved Registers

	Caller-Save	Callee-Save
Who saves?	The **caller** (before call)	The **callee** (at entry)
Destroyed by call?	Yes — assumed clobbered	No — must be preserved
Best for…	Vars **not live across calls**	Vars **live across calls**
In the graph	CALL defines them → interference with call-live vars	Copied to temps at entry; restored at exit

Both categories are modeled as **precolored nodes**. The key difference is how CALL instructions interact with them in the interference graph.

Modeling Caller-Save Registers

- In the IR, every CALL instruction is annotated to **define (clobber)** all caller-save registers.

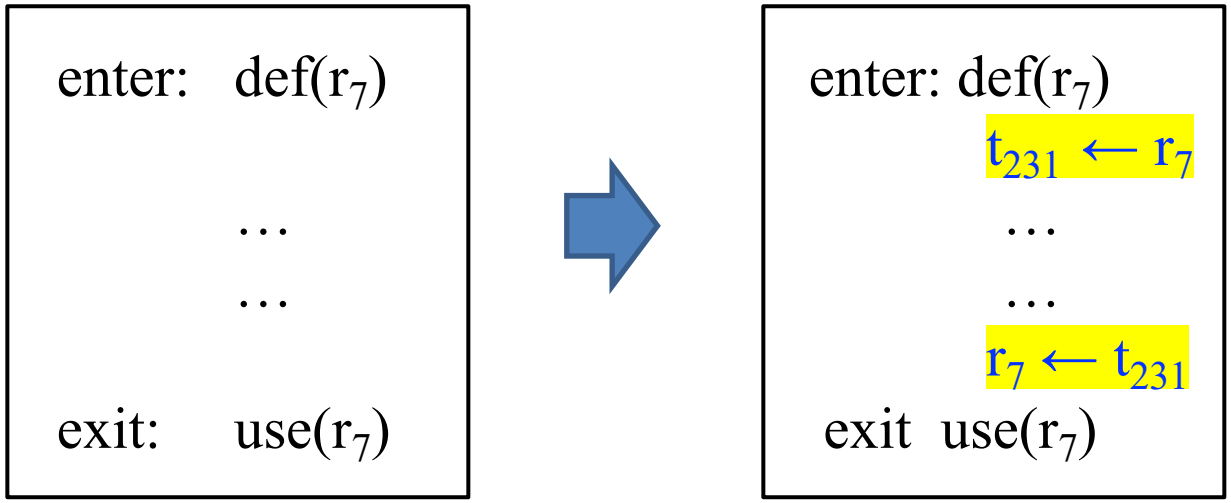
```
// CALL defines {r1, r2, r3} (caller-save set)
// If variable x is live across the call:
//    → x interferes with r1, r2, r3
//    → x will NOT be allocated to a caller-save reg ✓
```

- Effect on the interference graph:
  - Creates interference edges between **caller-save registers** and **every variable live across the call**
  - Variables **not** live across calls → no interference → naturally allocated to caller-save regs (no save/restore needed!)

Result: The graph coloring allocator **automatically** avoids assigning caller-save registers to variables that span calls — no special logic needed!

Modeling Callee-Save Registers

- Callee-save registers must be **preserved** by the function. We model this with MOVEs at entry/exit:



Long live range of r7

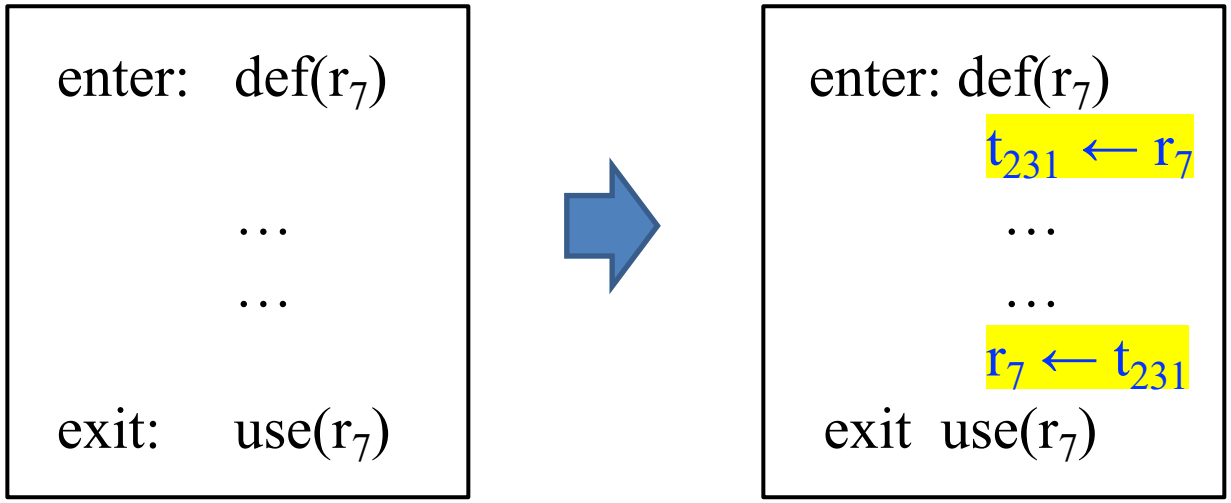
Split: short r_7, long t_{231}

What happens to t_{231}

- **Low register pressure** → t_{231} is coalesced with r_7 → MOVEs eliminated, zero cost
- **High register pressure** → t_{231} is *spilled to memory* → effectively saves/restores r_7 on the stack

Modeling Callee-Save Registers

- Callee-save registers must be **preserved** by the function. We model this with MOVEs at entry/exit:



Long live range of r7

Split: short r_7, long t_{231}

The allocator **automatically decides** whether to actually save callee-save registers — only when needed!!

Short Summary

Concept	Key Takeaway
Precolored Nodes	Model hardware register constraints; cannot simplify or spill; infinite degree
George's Criterion	Safe coalescing test for moves involving precolored nodes
Short Live Ranges	Front end must MOVE values out of precolored regs quickly
Caller-Save	CALL defines them → auto-interference with cross-call variables
Callee-Save	Entry/exit MOVEs to fresh temps → allocator decides save vs. coalesce

The beauty of graph coloring: by encoding constraints as **interference edges** and **move edges**, the allocator handles precoloring and calling conventions **uniformly** within the same framework.

4. Coloring By Simplifications. Cont.

- Coalescing
- Precolored Nodes
- Putting it all Together

Put it all Together (K=3)

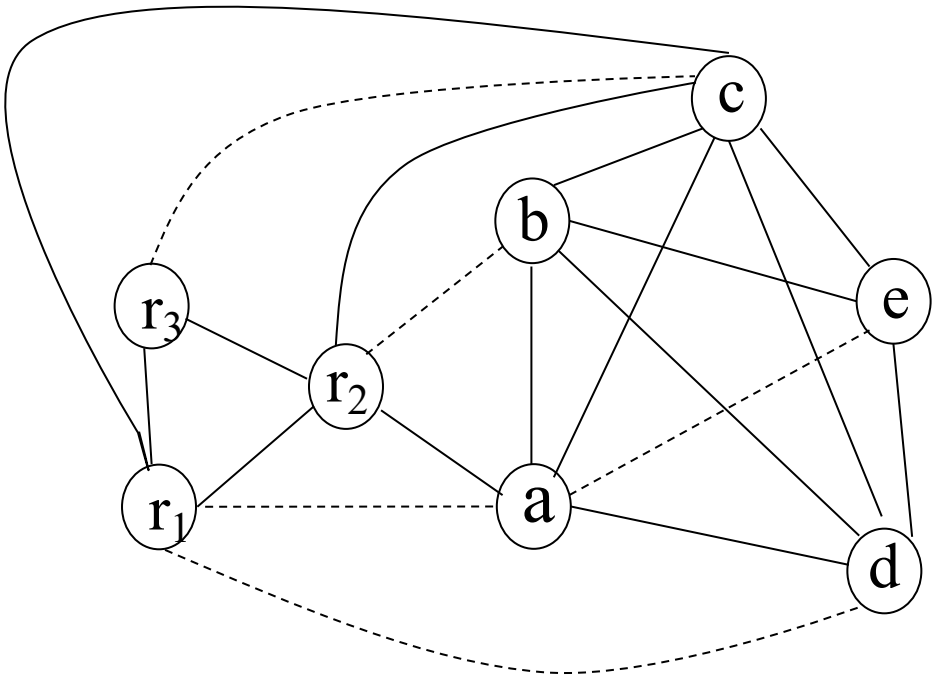
```
int f(int a, int b) {
  int d = 0;
  int e = a;
  do {
       d = d+b;
       e = e-1;
  } while (e>0);
  return d;
}
```

```
enter:  c←r3   // Save callee-save r3
        a ← r1 // Parameter in r1
        b ← r2 // Parameter in r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e – 1
        if (e > 0) goto loop
        r1 ← d
        r3 ← c  // restore r3
        return  (r1, r3 live out)
```

Assumption: a machine with 3 registers:

- **r1 and r2 are caller-save**
- **r3 is callee save**

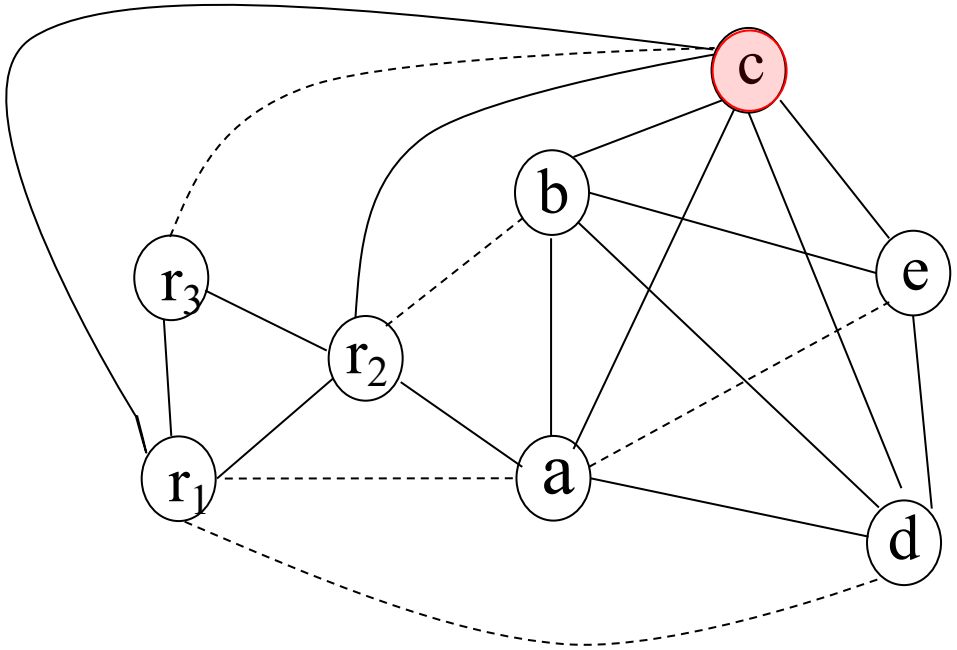
Put it all Together (K=3)



- **No opportunity to simplify or freeze**
  - All the non-precolored nodes have degree >= K
- **No way to coalesce**
  - No less than K significant-degree nodes for coalesced nodes
- **The only way is spilling**

(choose a potential actual spiling node)

Spilling Criteria: Grade Non-Precolored Nodes




```
enter:   c←r3
         a ← r1
         b ← r2
         d ← 0
         e ← a
loop:    d ← d + b
         e ← e – 1
         if (e > 0) goto loop
         r1 ← d
         r3 ← c
         return   (r1, r3 live out)
```

Node	Use+Def Outside loop	Use+Def inside loop	Node Degree	Spill priority
a	2	0	4	(2+10*0)/4=0.5
b	1	1	4	(1+10*1)/4=2.75
c	2	0	6	(2+10*0)/6=0.33
d	2	2	4	(2+10*2)/4=5.5
e	1	3	3	(1+10*3)/3=10.33

Spilling Criteria: The Spill Priority Formula

$$\text{Spill Priority} = \frac{\text{uses+defs}_{\text{outside}} + 10 \times \text{uses+defs}_{\text{inside loop}}}{\text{degree}}$$

- **Uses+Defs outside loop** — cost of spilling outside loops
- **Uses+Defs inside loop** — cost inside loops (**×10 weight**)
- Degree — # of interferences removed by spilling

⬇️ **Lower priority → Better spill candidate** ⬇️

The Spill Priority Formula: Why This Works

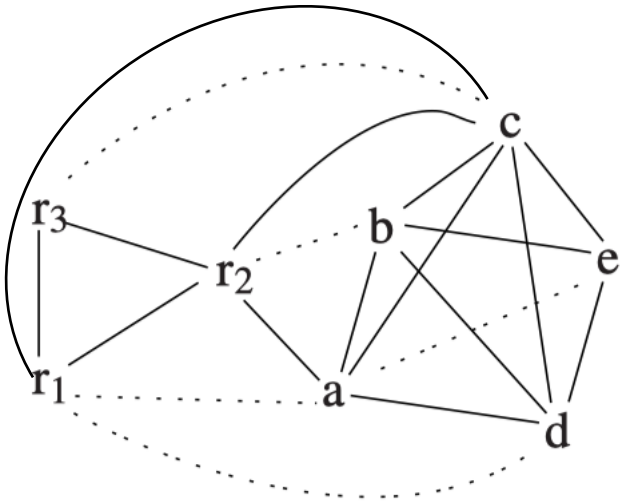
$$\text{Spill Priority} = \frac{\text{uses+defs}_{\text{outside}} + 10 \times \text{uses+defs}_{\text{inside loop}}}{\text{degree}}$$

- **Numerator = Cost of Spilling**
  - Each use/def needs a LOAD/STORE
  - Inside a loop → executed many times
  - Weight factor 10 approximates loop iteration count
  - **High cost → Don't want to spill**
- **Denominator = Benefit of Spilling**
  - Removing a high-degree node frees many neighbors
  - More neighbors freed → easier to color the rest
  - **High degree → Good candidate**

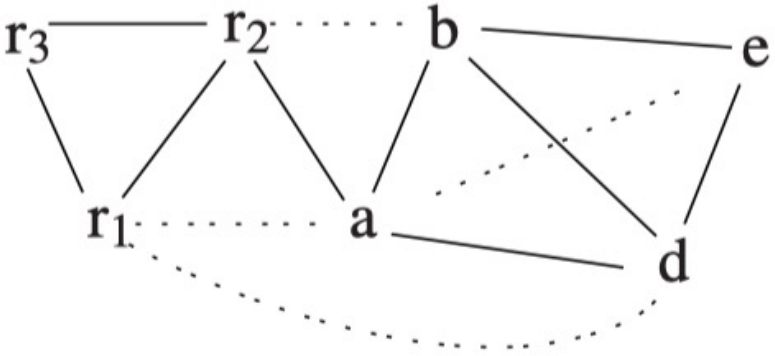
Spill Priority =

Cost / Benefit → **minimize** to pick the best candidate

Step 1. (Potentially) Spill Node c

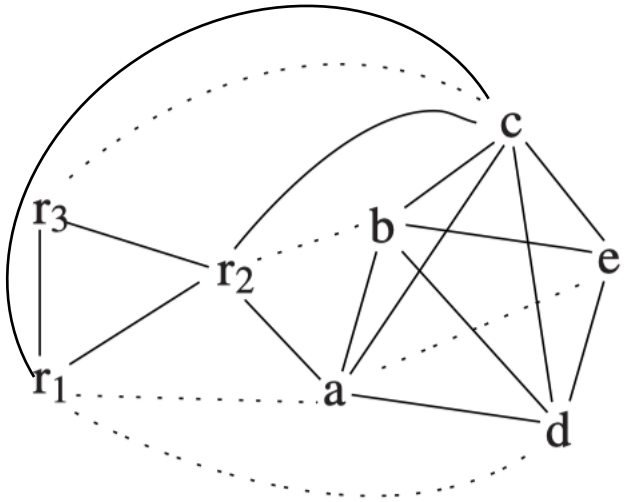


1. Spill node c

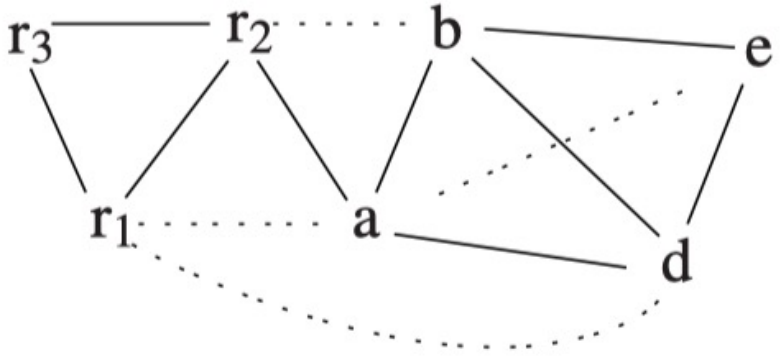


- Spill node c
- No simplify is possible
  - All non-precolored nodes are move-related

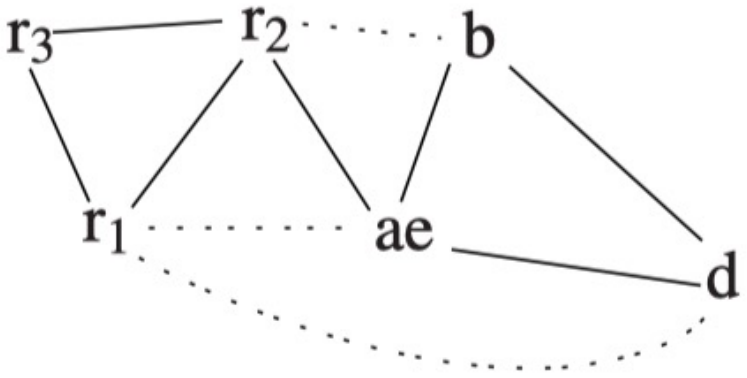
Step 2. Coalesce a & e



1. Spill node c

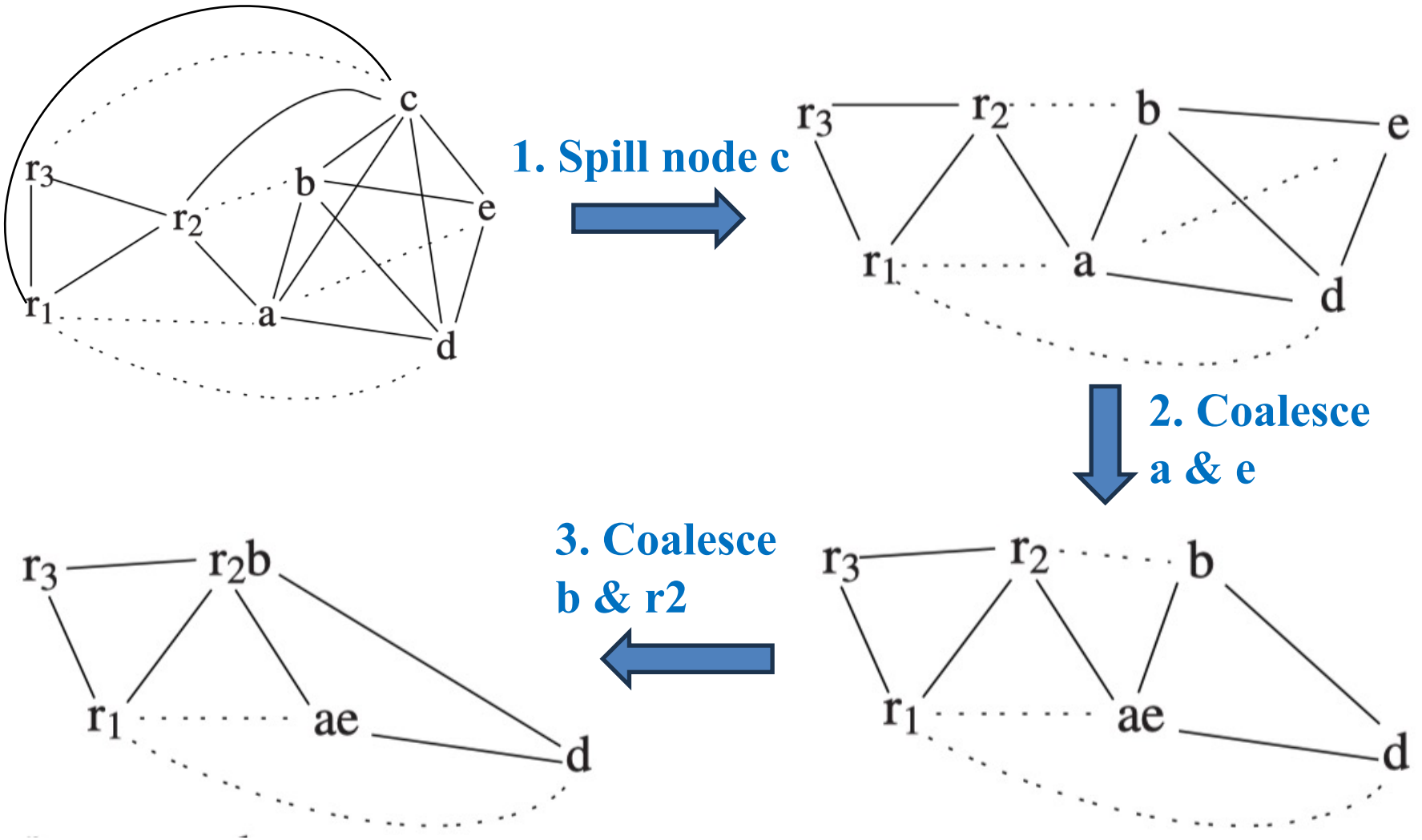


2. Coalesce a & e



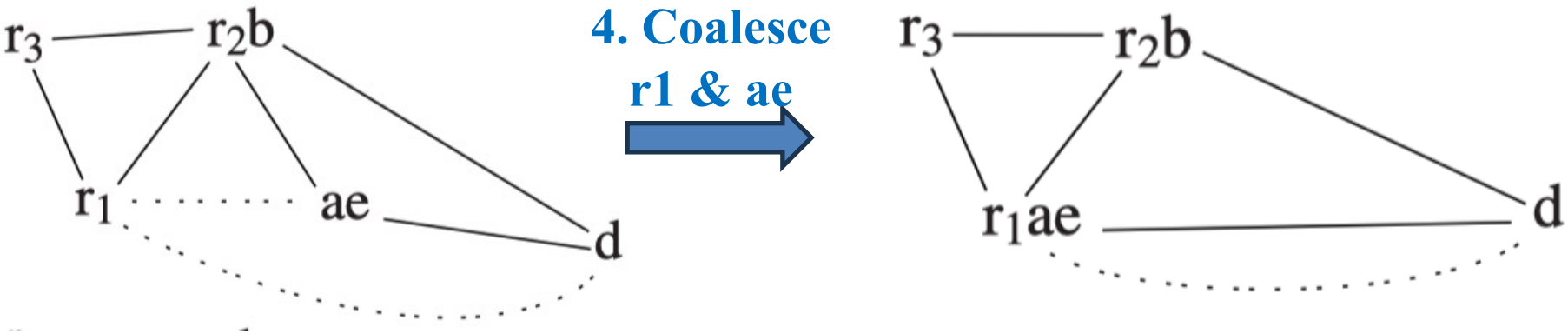
- Precolored节点时，使用George策略
- 不是precolored节点时，使用Briggs策略

Step 3. Coalesce b & r2



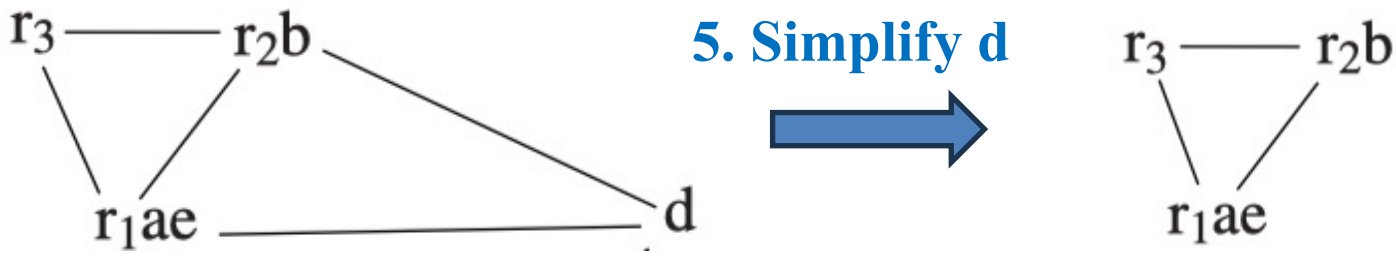
- Perform coalescing

Step 4. Coalesce r1 & ae



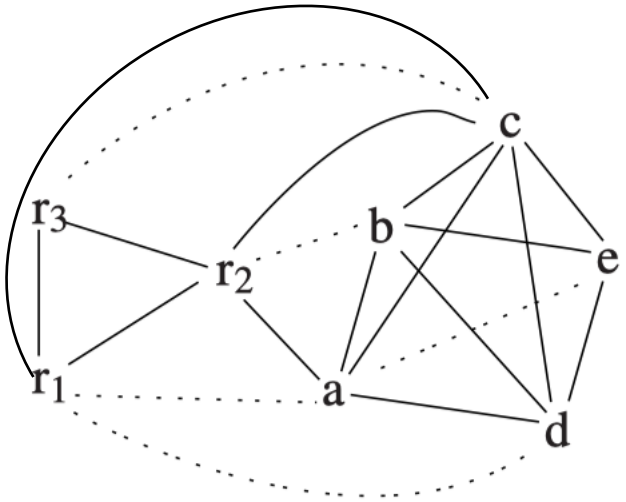
- Perform coalescing
- Now, can we coalesce r1ae and d?
  - No, r1ae interferes with d
- The move between r1ae and d is **constrained move**
  - We remove it from further considerations
  - **d is no longer treated as move-related**

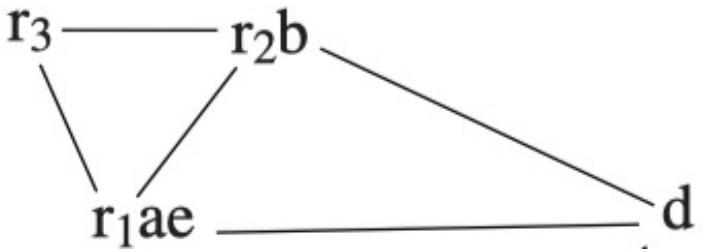
Step 5. Simplify d



- We must simplify d
- After this step, only precolored nodes left!

Step 6. Select





d
c

- Pop nodes from the stack and assign color to them:
  - Pick d, assign color r3
  - Nodes a, b, e have already been assigned colors by coalescing
  - Pop c: c turns into an actual spill (Why?)

Step 7. Rewrite (Do the Actual Spill)

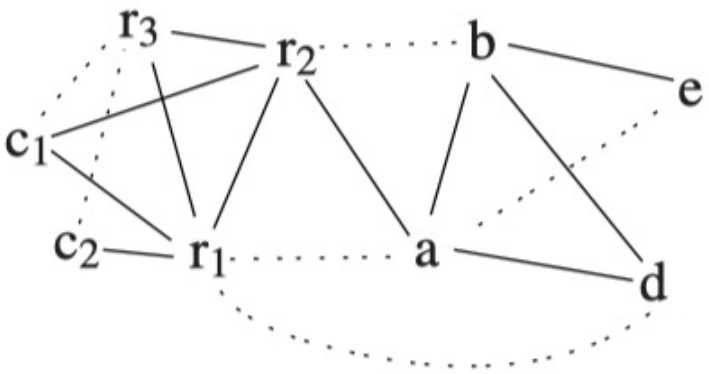
- There is spilling, we must rewrite the program to include spill instructions.
  1. Before each use -> fetch
  2. After each def -> store

```
enter:  c←r3
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e – 1
        if (e > 0) goto loop
        r1 ← d
        r3 ← c
        return
```

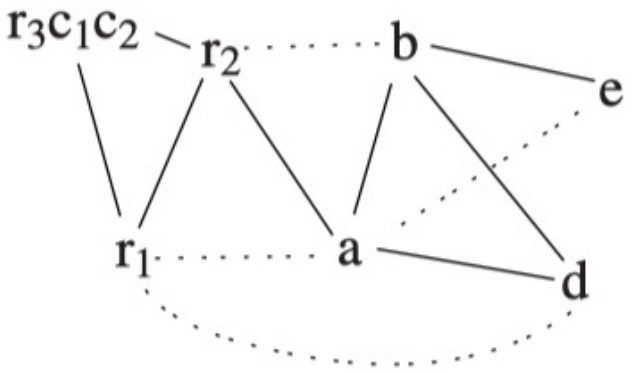
rewrite

```
enter:  c1 ←r3
        M[cloc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e – 1
        if (e > 0) goto loop
        r1 ← d
        c2 ← M[cloc]
        r3 ← c2
        return
```

Stpes 8, 9, 10, 11: Starting Over (A New Round)

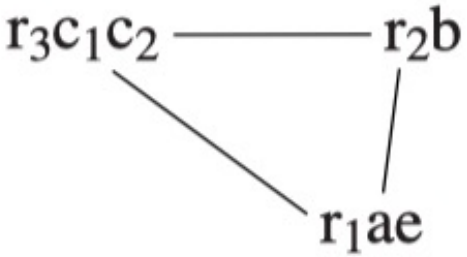


9. coalesce c1&r3
and then c2&r3

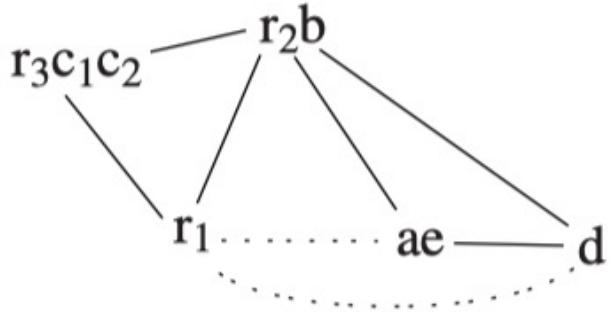


8. Build a new inference graph

10. coalesce a&e
and then b&r2

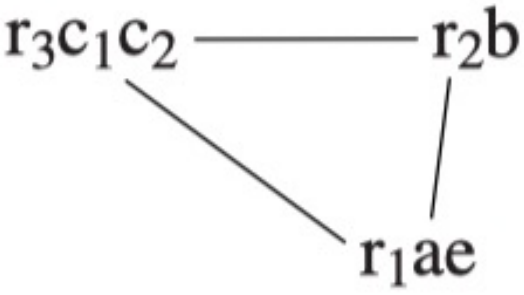


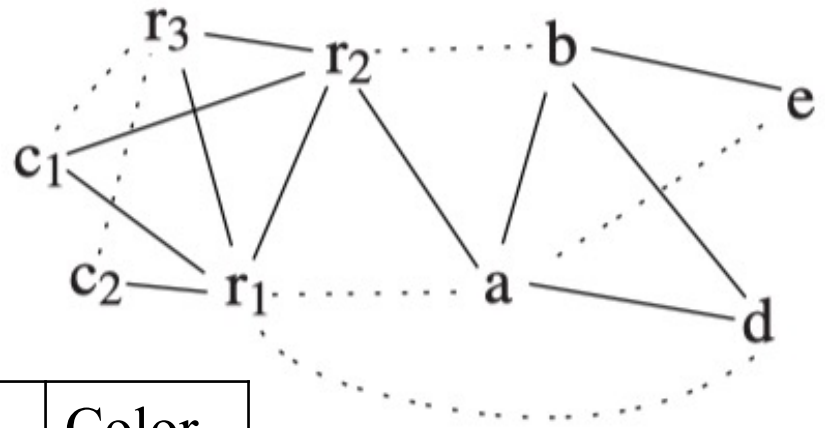
11. coalesce ae&r1
and then simplify d

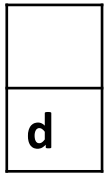


注: 这里的“Step”只是泛称，可能合并了一些具体步骤

Step 12: Select







Node	Color
a	r1
b	r2
c	r3
d	r3
e	r1

- Poping from the stack and select color r3 for d
- all other nodes were coalesced or precolord

After the Register Assignments

Step 13. Rewrite the program using the register assignment

```
enter:  c1 ←r3
        M[cloc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e – 1
        if (e > 0) goto loop
        r1 ← d
        c2 ← M[cloc]
        r3 ← c2
        return
```

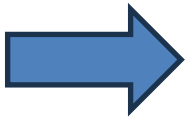
Node	Color
a	r1
b	r2
c	r3
d	r3
e	r1

```
enter:  r3 ←r3
        M[cloc] ← r3
        r1 ← r1
        r2 ← r2
        r3 ← 0
        r1 ← r1
loop:   r3 ← r3 + r2
        r1 ← r1 – 1
        if (r1 > 0) goto loop
        r1 ← r3
        r3 ← M[cloc]
        r3 ← r3
        return
```

After the Register Assignments

Step 14. Delete any move instruction whose source and destination are the same

```
enter:  r3 ←r3
        M[c_loc] ← c_1
        r1 ← r1
        r2 ← r2
        r3 ← 0
        r1 ← r1
loop:   r3 ← r3 + r2
        r1 ← r1 – 1
        if (r1 > 0) goto loop
        r1 ← r3
        r3 ← M[c_loc]
        r3 ← r3
        return
```



```
enter:  M[c_loc] ← r_3
        r3 ← 0
loop:   r3 ← r3 + r2
        r1 ← r1 – 1
        if (r1 > 0) goto loop
        r1 ← r3
        r3 ← M[c_loc]
        return
```

After the Register Assignments

14. Delete any move instruction whose source and destination are the same

```
enter:  M[c_loc] ← r3          // save callee-save r3
        r3 ← 0                 // d in r3
loop:   r3 ← r3 + r2           // d = d + b
        r1 ← r1 – 1            // e = e - 1
        if (r1 > 0) goto loop
        r1 ← r3                // return value = d
        r3 ← M[c_loc]          // restore callee-save r3
        return
```

- Only **1 spill** (callee-save r3, saved/restored via stack) — necessary!
- **5 out of 6 MOVEs eliminated** by coalescing
- Tight, efficient loop body with no unnecessary moves

Summary

- Register allocation maps temporaries to a limited set of machine registers
- Graph coloring provides an effective approach to global register allocation
- Conservative coalescing eliminates most redundant MOVE instructions (Briggs and George criteria)
- Spilling allows compilation even when there aren't enough registers
- Precolored nodes handle machine registers used for calling conventions

Thank you all for your attention

一点补充

关于Actual Spill

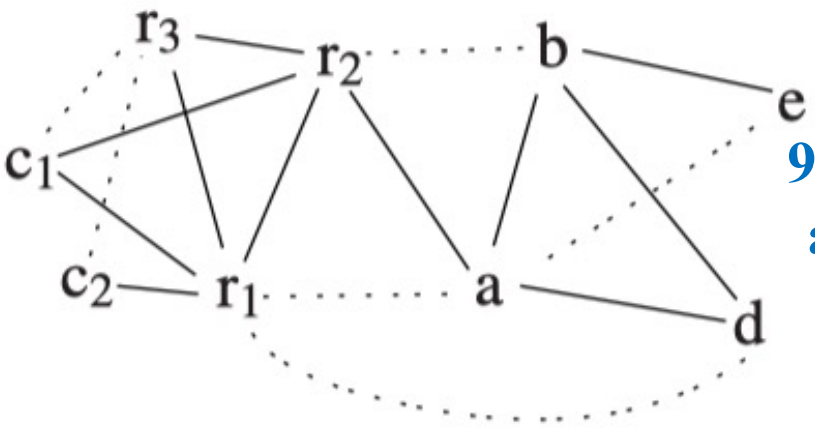
Tiger book的寄存器分配中，是否可以一次性spill多个节点，还是说发现多个acctual spill后选一个每次只actual spill一个？

- 1. 选“潜在 spill”时（第一个大步骤“Simplify”）——一次只选一个，用启发式的spilig cost
- 2. 在select阶段可能累积得到多个 “actual spill”,但是其中一些可能因为Optimisc策略而不用针的do actual spill。
- 3. Select阶段最终可能剩下不止一个actal spill。 那么 Rewrite——把所有 actual spill 一次性插入内存访问，再重跑整个分配器。

==>

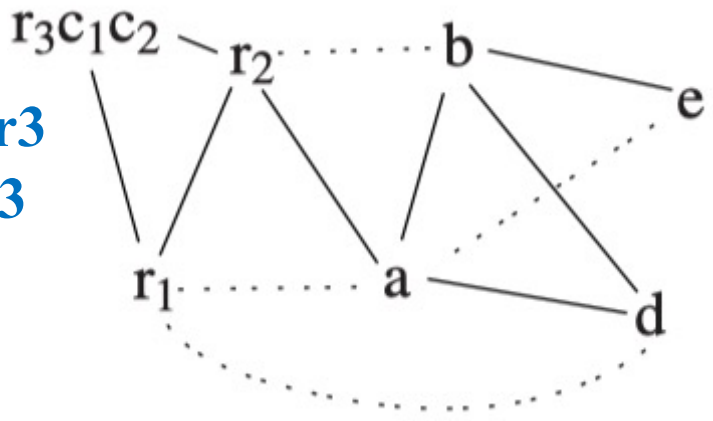
不过如果题目让你”算spilling cost，然后选一个马上做actual spilling（也就是rewrite program)", 那就按照题目说的去做吧

About 8, 9, 10, 11: Starting Over (A New Round)

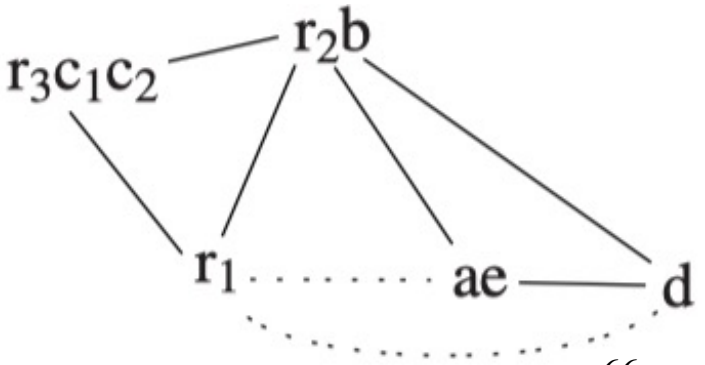


8. Build a new inference graph

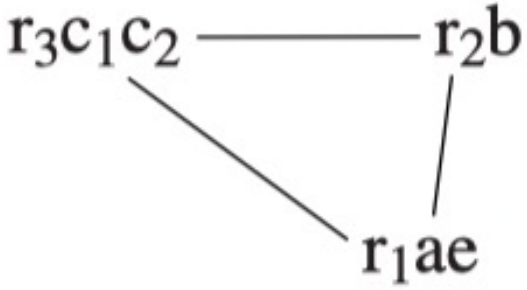
9. coalesce c1&r3 and then c2&r3



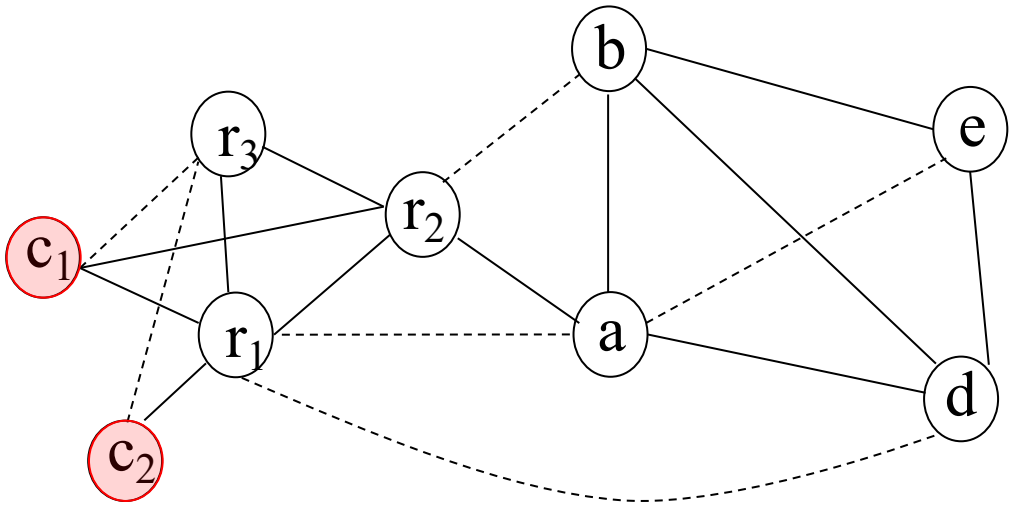
10. coalesce a&e and then b&r2



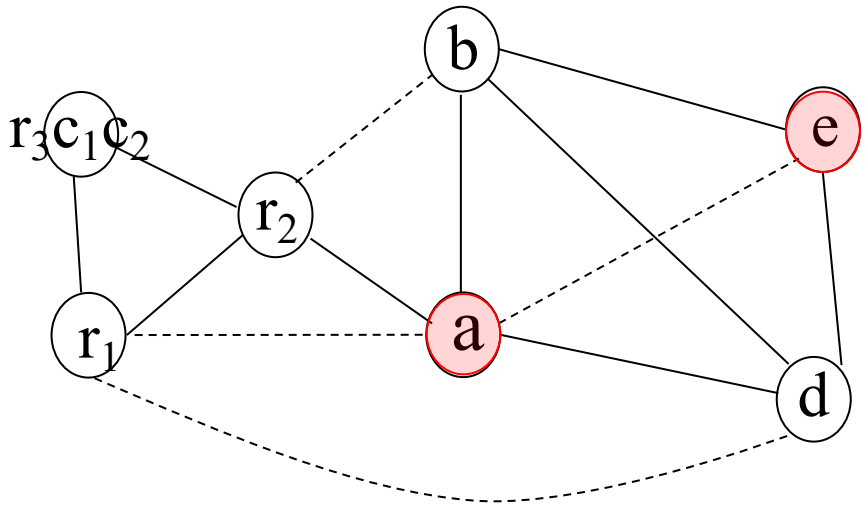
11. coalesce ae&r1 and then simplify d



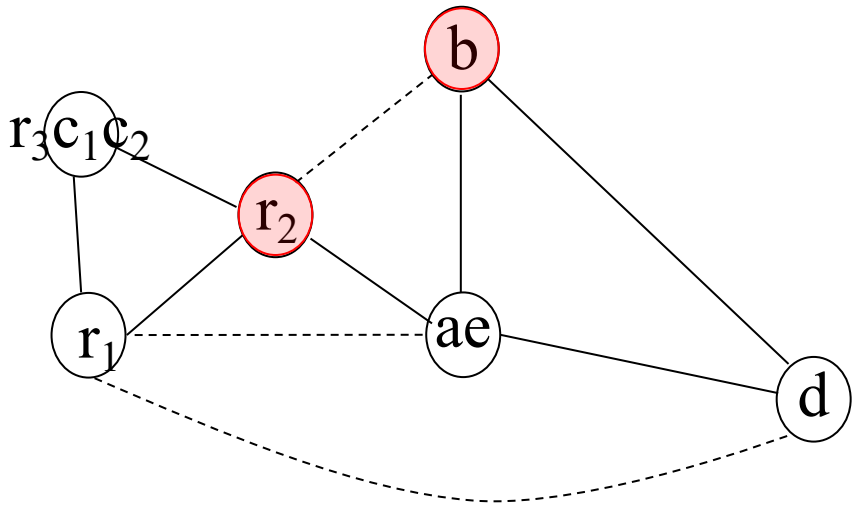
```
enter:  c1 ← r3
        M[c_loc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d+b
        e ← e-1
        if e>0 goto loop
        r1 ← d
        c2 ← M[c_loc]
        r3 ← c2
        return (r1, r3 live out)
```



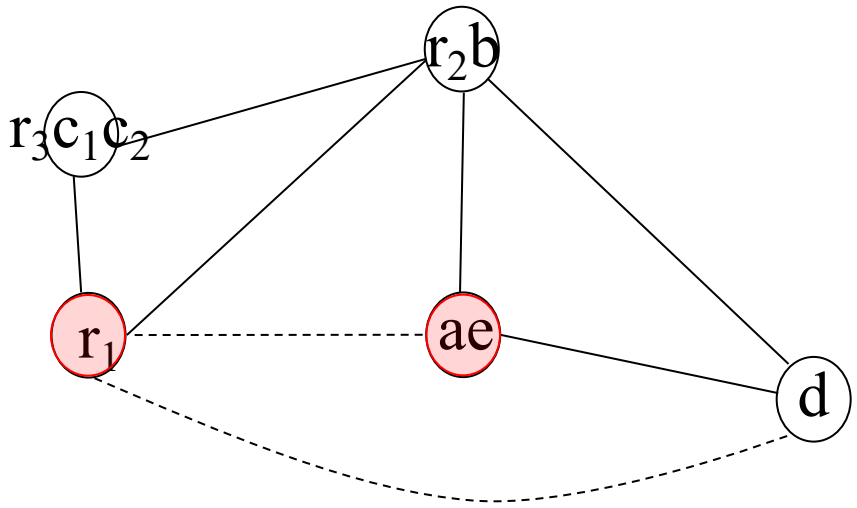
```
enter:  c1 ← r3
        M[c_loc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
        loop:   d ← d+b
        e ← e-1
        if e>0 goto loop
        r1 ← d
        c2 ← M[c_loc]
        r3 ← c2
        return (r1, r3 live out)
```



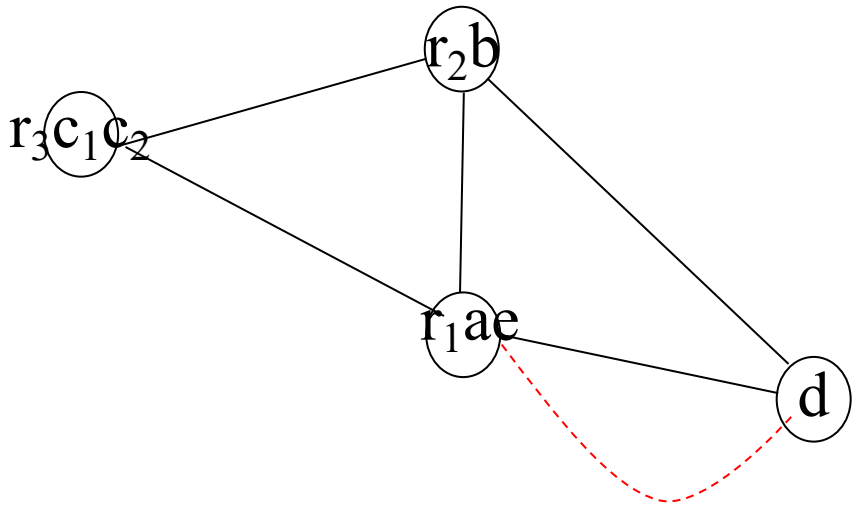
```
enter:  c1 ← r3
        M[c_loc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
        loop:   d ← d+b
        e ← e-1
        if e>0 goto loop
        r1 ← d
        c2 ← M[c_loc]
        r3 ← c2
        return (r1, r3 live out)
```



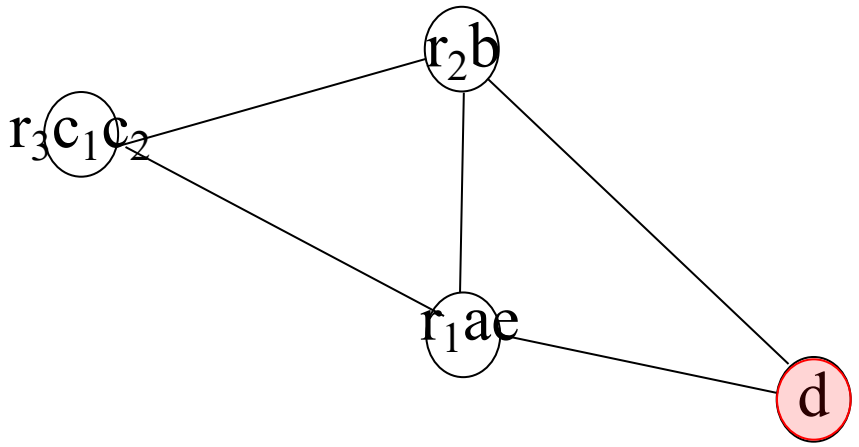
```
enter:  c1 ← r3
        M[c_loc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
        loop:   d ← d+b
        e ← e-1
        if e>0 goto loop
        r1 ← d
        c2 ← M[c_loc]
        r3 ← c2
        return (r1, r3 live out)
```



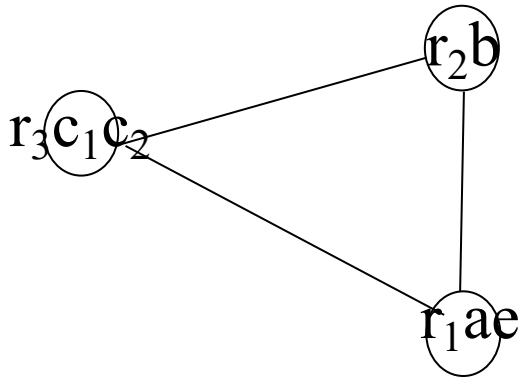
```
enter:  c1 ← r3
        M[c_loc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
        loop:   d ← d+b
        e ← e-1
        if e>0 goto loop
        r1 ← d
        c2 ← M[c_loc]
        r3 ← c2
        return (r1, r3 live out)
```



```
enter:  c1 ← r3
        M[c_loc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
        loop:   d ← d+b
        e ← e-1
        if e>0 goto loop
        r1 ← d
        c2 ← M[c_loc]
        r3 ← c2
        return (r1, r3 live out)
```



```
enter:  c1 ← r3
        M[c_loc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
        loop:   d ← d+b
        e ← e-1
        if e>0 goto loop
        r1 ← d
        c2 ← M[c_loc]
        r3 ← c2
        return (r1, r3 live out)
```



```
d       r3
a       r1
e       r1
b       r2
c       r3
enter:  M[c_loc] ← r3
        r3 ← 0
        loop:   r3 ← r3+r2
        r1 ← r1-1
        if r1>0 goto loop
        r1 ← r3
        r3 ← M[c_loc]
        return
```